



ESI – École nationale Supérieure d'Informatique

SIQ – Base de données avancées

Moteur de recherche sémantique et analytique sur des logs massifs Big Data

Rapport technique

Auteurs	Tati Youcef – my_tati@esi.dz Mostefai Mounir Sofiane – mm_mostefai@esi.dz
Encadrement	Ens. Amrouche Karima – k_amrouche@esi.dz

Année universitaire 2025–2026

Contents

1	Introduction	1
2	Jeu de donnees	1
3	Architecture generale	2
4	Pretraitement Big Data	2
5	Schema de stockage	3
6	Details d'implementation	3
7	Choix de conception	4
8	Vectorisation	4
9	Recherche et analyse	5
9.1	Recherche semantique	5
9.2	Recherche par mots-cles	5
9.3	Logs similaires	5
9.4	Analytique	5
10	Interface de visualisation	6
11	Reproductibilite	6
12	API applicative	6
13	Evaluation des performances et comparaison de modeles	7
14	Cas d'utilisation observes	8
15	Validation des resultats	8
16	Tests et validation	10
17	Limites et ameliorations	10
18	Conclusion	11

Abstract

Ce rapport présente la conception et la réalisation d'une plateforme Big Data de recherche sémantique appliquée aux logs OpenSSH de LogHub-2.0. La solution combine Apache Spark pour le prétraitement batch, PostgreSQL avec pgvector pour le stockage vectoriel indexé, Sentence-Transformers pour la représentation sémantique, FastAPI pour l'exposition des services et Streamlit pour la visualisation interactive. Le système traite 638 947 entrées, fournit une recherche sémantique, une recherche lexicale de référence, l'identification de groupes d'erreurs récurrentes, l'analyse temporelle et une évaluation comparative de plusieurs modèles d'embeddings.

Mots-clés : Big Data, logs, recherche sémantique, pgvector, Apache Spark, Sentence-Transformers, analyse d'erreurs.

1 Introduction

Les infrastructures modernes produisent des volumes importants de journaux d'exécution. Dans un système distribué, ces journaux proviennent de machines, services, processus et utilisateurs différents. Leur exploitation par recherche textuelle simple reste limitée : une même panne peut être formulée par plusieurs messages proches, et une requête par mot-clé peut ignorer des logs pertinents lorsque les termes exacts ne sont pas présents.

Cette étude présente une plateforme Big Data capable d'ingérer un grand volume de logs, de les structurer avec Apache Spark, de transformer les messages en vecteurs sémantiques, de stocker ces vecteurs dans PostgreSQL avec pgvector, puis de fournir des fonctions de recherche et d'analyse. L'analyse porte sur une chaîne complète, reproductible et observable, depuis le traitement batch jusqu'aux résultats consultables par API et interface web.

La solution s'appuie exclusivement sur des composants open source : Python, Apache Spark, PostgreSQL, pgvector, Sentence-Transformers, FastAPI et Streamlit. Le rapport décrit le jeu de données, les choix de conception, le schéma de stockage, les fonctions de recherche et les résultats observés.

2 Jeu de données

Le dataset utilisé est OpenSSH de LogHub-2.0. Il contient 638 947 entrées brutes dans l'extraction locale, ce qui fournit un volume suffisant pour évaluer un pipeline de recherche sur logs massifs. Le dépôt local contient trois fichiers principaux :

Fichier	Rôle
OpenSSH_full.log	Logs bruts au format syslog, avec mois, jour, heure, hôte, service et message.
OpenSSH_full.log_structured.csv	Version structurée fournie par LogHub : identifiant de ligne, contenu, identifiant d'événement et template.
OpenSSH_full.log_templates.csv	Liste des templates d'événements et leurs occurrences.

Un exemple de ligne brute est :

```
Dec 10 06:55:48 LabSZ sshd[24200]: Failed password for invalid user webmaster
from 173.234.31.186 port 38926 ssh2
```

La version structuree separe le contenu metier du prefixe syslog. Elle fournit aussi un `EventId` et un `EventTemplate`. Ces templates sont importants car ils reduisent le bruit des valeurs variables comme adresses IP, ports, noms d'utilisateurs et identifiants de processus.

3 Architecture generale

L'architecture suit une chaine batch classique adaptee a un contexte Big Data :

```
Logs OpenSSH
-> Pretraitement Spark
-> CSV/Parquet traites
-> PostgreSQL + pgvector
-> API FastAPI
-> Interface Streamlit
```

Spark est utilise pour charger les fichiers volumineux, parser les prefixes syslog, joindre le log brut avec le CSV structure, normaliser les messages et produire des sorties persistantes. PostgreSQL stocke les logs nettoyes et une table vectorielle des messages normalises distincts. pgvector ajoute le type `vector(384)` et l'index HNSW pour la recherche approximative de voisins proches. FastAPI expose les fonctionnalites sous forme d'API HTTP, et Streamlit fournit la visualisation interactive.

La separation entre pipeline, base, API et interface rend la solution reproductible : le pipeline peut etre lance en ligne de commande, l'API peut etre testee seule, et l'interface consomme uniquement les endpoints publics.

4 Pretraitement Big Data

La commande `tp5-log-search prepare-data` execute le pretraitement Spark. Le traitement lit le fichier brut, associe un numero de ligne, puis joint ce flux avec le CSV structure par `LineId`. Cette strategie permet de conserver les informations temporelles du log brut et les templates fournis par LogHub.

Les champs extraits sont :

- `line_id`, identifiant stable de la ligne ;
- `log_timestamp`, timestamp reconstruit avec une annee configurable ;
- `host`, `service`, `process_id` ;
- `level`, niveau heuristique : INFO, WARNING, ERROR ou CRITICAL ;
- `raw_message`, message original ;
- `normalized_message`, message normalise vectorise ;

- `event_id` et `event_template`.

Le fichier OpenSSH ne contient pas l'annee. Pour permettre les analyses temporelles, le pipeline utilise une variable `LOG_YEAR`, fixee par default a 2024. Cette hypothese est explicite, configurable et ne modifie pas l'ordre relatif des evenements.

Le resultat est ecrit sous deux formats. Le CSV sert au chargement rapide dans PostgreSQL avec `COPY`. Le Parquet conserve une sortie analytique compacte et partitionnee par niveau de log, utile pour des analyses Spark supplementaires.

5 Schema de stockage

Le schema relationnel est centre sur trois tables :

Table	Description
<code>event_templates</code>	Dictionnaire des templates LogHub et nombre d'occurrences annonce.
<code>log_entries</code>	Lignes de logs nettoyees, metadonnees temporelles, niveau et message.
<code>message_embeddings</code>	Messages normalises distincts, occurrences et vecteurs semantiques indexes.
<code>pipeline_runs</code>	Trace d'execution du pipeline complet.

La colonne vectorielle principale est stockee dans `message_embeddings` :

```
embedding VECTOR(384)
```

La dimension 384 correspond au modele `sentence-transformers/all-MiniLM-L6-v2`. Les index relationnels couvrent le niveau, le temps, l'evenement et la recherche textuelle par trigrammes. La recherche semantique repose sur :

```
CREATE INDEX message_embeddings_embedding_hnsw_idx
ON message_embeddings USING hnsw (embedding vector_cosine_ops);
```

L'index HNSW est adapte a la recherche de similarite a grande echelle car il evite un parcours sequentiel complet pour chaque requete. La distance utilisee est le cosinus, coherente avec les embeddings normalises.

6 Details d'implementation

Le code source est organise comme un package Python installable. Cette organisation evite les scripts isoles difficiles a maintenir et donne une base logicielle claire et modulaire. Les modules principaux sont separes par responsabilite : configuration, pretraitement Spark, acces base de donnees, embeddings, recherche, analytique, API, interface et CLI.

Le module de configuration lit les variables d'environnement depuis `.env`. Les chemins sont resolus relativement a la racine du depot afin que les commandes fonctionnent depuis

n'importe quel repertoire. Les valeurs par default pointent vers le dataset OpenSSH deja extrait dans `data/raw/OpenSSH`.

Le module Spark evite de supposer que le CSV structure contient toutes les informations utiles. Il reconstruit les metadonnees systeme depuis le log brut, puis joint ces informations avec les templates LogHub. Cette decision conserve a la fois la richesse temporelle du fichier brut et la qualite du parsing fourni par LogHub.

Le chargement PostgreSQL utilise `COPY` plutot que des insertions ligne par ligne. Pour un dataset de plusieurs centaines de milliers de lignes, cette difference est importante : `COPY` reduit le temps de chargement et limite la charge cote client Python. Les commandes peuvent etre relancees car le pipeline initialise le schema puis recharge les tables applicatives de maniere deterministe.

La recherche semantique est encapsulee dans un module dedie. L'API et l'interface n'ont donc pas a connaitre les details SQL de pgvector. Cette separation facilite les tests et permettrait de changer le modele d'embedding ou la base vectorielle sans reecrire l'interface.

7 Choix de conception

Plusieurs choix ont ete faits pour concilier performance locale, reproductibilite et clarte operationnelle. Le premier choix est d'utiliser PostgreSQL comme base unique pour les donnees relationnelles et vectorielles. Cette approche simplifie l'exploitation : les filtres temporels, niveaux de severite, templates et vecteurs sont interroges dans le meme systeme.

Le deuxieme choix est de vectoriser les messages normalises plutot que chaque message brut independamment. Sur un dataset de logs structures, les templates representent le sens operationnel des evenements, tandis que les valeurs variables representent souvent des parametres. Cette vectorisation rend les groupes d'erreurs plus stables et accelere fortement l'indexation.

Le troisieme choix est de conserver une recherche par mots-cles. Elle ne remplace pas la recherche semantique, mais elle reste pertinente pour des investigations precises. Par exemple, une adresse IP ou un port doit etre retrouve lexicalement. La comparaison entre les deux methodes montre donc leur complementarite.

Le dernier choix est de fournir une API en plus de l'interface. Cette separation explicite le contrat applicatif et evite de coupler l'interface utilisateur aux details SQL. Avec FastAPI, la interface web, les tests et d'eventuels clients externes consomment le meme contrat.

8 Vectorisation

La vectorisation est realisee avec Sentence-Transformers. La commande `tp5-log-search embed` charge le modele, recupere les messages normalises distincts non encore vectorises, calcule les embeddings par lots, puis les insere dans la table `message_embeddings`.

Cette approche est volontaire : la table vectorielle ne contient pas un vecteur par ligne brute, mais un vecteur par message normalise distinct, generalement issu du template LogHub. Les lignes brutes restent toutes conservees dans `log_entries` et gardent leurs timestamps, niveaux, identifiants, hote et message original. La liaison entre une ligne et

son vecteur se fait par `normalized_message`.

Ce choix est adapté aux logs structurés. Dans OpenSSH, beaucoup de lignes partagent le même template et ne diffèrent que par des valeurs variables comme l'adresse IP, le port ou le nom d'utilisateur. Calculer et stocker un embedding dupliqué pour chaque ligne brute gaspillerait du temps CPU et de l'espace disque sans ajouter beaucoup de signal sémantique. En vectorisant les templates normalisés, les 638 947 logs restent recherchables par jointure avec une table vectorielle compacte et indexée, tout en conservant la possibilité d'afficher chaque occurrence originale.

Les vecteurs sont normalisés côté modèle. La similarité retournée par l'API est :

```
1 - (embedding <=> query_embedding)
```

ou `<=>` est l'opérateur de distance cosinus de pgvector.

9 Recherche et analyse

La plateforme expose quatre familles de fonctionnalités.

9.1 Recherche sémantique

L'utilisateur formule une requête en langage naturel, par exemple :

```
brute force ssh authentication failure
```

Le système vectorise cette requête, interroge pgvector et retourne les logs les plus proches. Cette recherche peut retrouver des messages pertinents même si les mots exacts ne sont pas présents.

9.2 Recherche par mots-cles

La recherche par mots-cles utilise ILIKE et l'index trigramme. Elle sert de baseline pour comparer les résultats avec la recherche sémantique. Cette comparaison met en évidence les limites d'une recherche lexicale stricte sur des logs variables.

9.3 Logs similaires

L'endpoint `/logs/{id}/similar` récupère le vecteur d'un log déjà indexé, puis cherche ses voisins les plus proches. Cette fonction facilite l'investigation d'incidents en retrouvant les logs similaires à une erreur critique donnée.

9.4 Analytique

Les groupes d'erreurs fréquentes sont calculés par agrégation sur `event_id`, `event_template` et `level`. L'évolution temporelle agrège les logs par heure ou par jour. Lorsqu'une requête sémantique est fournie, le système identifie d'abord les événements les plus proches, puis calcule leur évolution sur l'ensemble de la base.

10 Interface de visualisation

L'interface Streamlit est organisée en cinq onglets : recherche, comparaison, logs similaires, analytique et benchmarks. Elle consomme l'API FastAPI et n'accède jamais directement à la base. Cette séparation est importante pour garder une architecture claire et testable.

La page d'accueil affiche les indicateurs principaux : nombre total de logs, couverture de vectorisation, nombre d'événements et période observée. Les tableaux de résultats affichent les identifiants de logs, timestamps, niveaux, événements, similarités et messages bruts. Les analyses de récurrences et temporelles utilisent des graphiques Plotly. L'onglet benchmarks complète l'observation du système avec les tailles de disque, la latence des requêtes, un score de cohérence top-k et une comparaison de plusieurs modèles Sentence-Transformers.

11 Reproductibilité

La plateforme est pilotée par une CLI unique :

```
tp5-log-search prepare-data
tp5-log-search init-db
tp5-log-search load-db
tp5-log-search embed
tp5-log-search index
tp5-log-search stats
tp5-log-search benchmark --query "failed password invalid user"
tp5-log-search compare-models --query "authentication failure"
```

Pour une exécution exploratoire, l'option `--limit` permet de traiter un sous-ensemble :

```
tp5-log-search run-pipeline --limit 10000
```

Pour le traitement complet, la commande sans limite traite les 638 947 logs :

```
tp5-log-search run-pipeline
```

La configuration est centralisée dans `.env`. Les chemins du dataset, l'URL PostgreSQL, le modèle d'embedding, l'année de log et les tailles de batch sont modifiables sans changer le code.

12 API applicative

L'API FastAPI constitue le contrat applicatif entre la base vectorielle et les clients. Elle a été conçue pour exposer des opérations simples, chacune correspondant à un cas d'utilisation de l'analyse de logs. La table suivante résume les endpoints principaux.

Endpoint	Fonction
GET /health	Vérification de disponibilité de l'API.
GET /stats	Statistiques globales : logs, vecteurs, événements, période, index.

GET /benchmark	Volumes, tailles disque et informations de benchmark globales.
POST /benchmark/query	Latence semantique/mots-cles et coherence top-k pour une requete.
POST /benchmark/models	Comparaison de plusieurs modeles semantiques sur les memes templates.
POST /search/semantic	Recherche par similarite vectorielle a partir d'une requete libre.
POST /search/keyword	Recherche lexicale par mots-cles pour comparaison.
POST /search/compare	Retour simultane des resultats semantiques et lexicaux.
GET /logs/{id}/similar	Voisins semantiques d'un log deja stocke.
GET /analytics/frequent-errors	Groupes d'erreurs les plus frequents.
GET /analytics/timeline	Evolution temporelle par requete, evenement ou niveau.

Le chargement du modele d'embedding est mis en cache cote API. Sans ce cache, chaque requete semantique rechargerait le modele depuis le disque, ce qui augmenterait fortement la latence. Le cache garde un seul objet `EmbeddingService` par processus FastAPI.

Les requetes applicatives acceptent un parametre `top_k`. Celui-ci borne le nombre de resultats retournes afin d'eviter des reponses trop volumineuses. Les filtres par niveau permettent de limiter la recherche a des logs critiques, erreurs, avertissements ou informations.

13 Evaluation des performances et comparaison de modeles

La solution integre un module d'evaluation des performances afin de quantifier son comportement. Les metriques exposees sont le nombre de logs, le nombre de messages vectorises distincts, la taille de la base PostgreSQL, la taille de la table de logs, la taille de l'index HNSW, la latence de recherche semantique, la latence de recherche par mots-cles et un score de coherence top-k.

La coherence top-k n'est pas une precision supervisee, car le dataset ne fournit pas de labels de pertinence pour chaque requete. C'est un indicateur proxy defini par :

$$\text{coherence} = 0.65 * \text{similarite_moyenne} + 0.35 * \text{part_evenement_dominant}$$

Il favorise donc les resultats a la fois proches de la requete et concentres autour du meme evenement LogHub. Dans ce contexte, cet indicateur complete l'observation qualitative des resultats retournes sans se substituer a une evaluation annotee.

Une comparaison de modeles complete cette evaluation. Les modeles compares sont `all-MiniLM-L6-v2`, `multi-qa-MiniLM-L6-cos-v1` et `all-mpnet-base-v2`. La comparaison est faite en memoire sur les memes messages normalises distincts, car `all-mpnet-base-v2` produit des vecteurs de dimension 768 alors que l'index pgvector de production est defini en `vector(384)` pour le modele MiniLM. Cette approche permet de comparer la coherence, la similarite moyenne, la latence et la dimension sans casser la base vectorielle indexee.

Modele	Dimension	Role dans l'évaluation
<code>all-MiniLM-L6-v2</code>	384	Modele de production, compact et compatible avec l'index pgvector.
<code>multi-qa-MiniLM-L6-cos-v1</code>	384	Modele optimise pour questions/reponses et similarite cosinus.
<code>all-mpnet-base-v2</code>	768	Modele plus lourd, souvent plus expressif mais plus couteux.

14 Cas d'utilisation observes

Le comportement de la plateforme est observe a travers quatre cas d'utilisation representatifs. Le premier concerne la disponibilite de la base vectorielle : apres le lancement de PostgreSQL avec Docker Compose et l'execution du pipeline, l'API expose les statistiques de la base, la couverture de vectorisation et la presence de l'index HNSW.

Le deuxieme cas pratique consiste a chercher une erreur critique. Une requete comme `possible break in ssh server` retourne les logs du template `POSSIBLE BREAK-IN ATTEMPT`. Le resultat prouve que la recherche n'est pas limitee a une correspondance exacte des mots.

Le troisieme cas pratique porte sur les erreurs frequentes. L'onglet analytique affiche les groupes dominants : echecs de mot de passe, utilisateurs invalides, fermetures de connexion et echecs d'authentification PAM. Ces groupes correspondent aux evenements recenses dans LogHub et donnent une vision synthetique du comportement du service SSH.

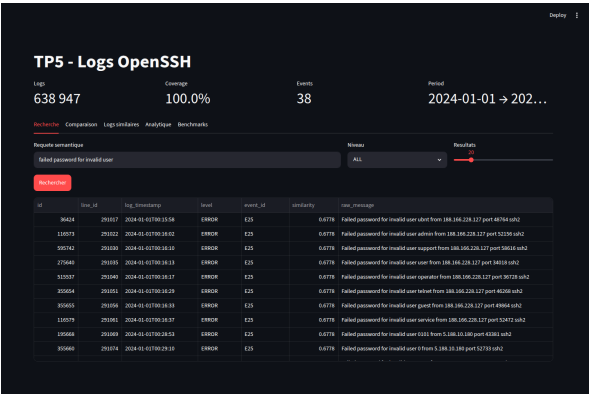
Le quatrieme cas pratique analyse l'evolution temporelle. A partir d'une requete semantique, l'application identifie les evenements proches puis agrege leurs occurrences par jour ou par heure. Cette vue permet de reperer des periodes de forte activite suspecte, par exemple une concentration d'echecs d'authentification.

L'onglet comparaison affiche cote a cote la recherche semantique et la recherche par mots-cles. Cette comparaison illustre l'interet des embeddings : la recherche semantique peut retrouver des messages proches meme lorsque le vocabulaire exact differe, tandis que la recherche par mots-cles reste utile pour des termes precis comme une adresse IP ou un nom d'utilisateur. L'onglet benchmarks montre ensuite les metriques de taille, latence, coherence top-k et comparaison des trois modeles semantiques.

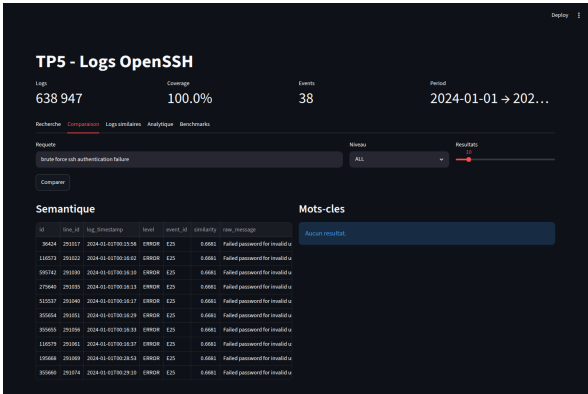
15 Validation des resultats

La validation combine des criteres fonctionnels, techniques et qualitatifs. Sur le plan fonctionnel, le systeme charge les donnees, remplit la base, vectorise les logs et retourne des resultats exploitables depuis l'API et l'interface. Sur le plan technique, le pipeline reste reproductible, le schema contient une colonne vectorielle indexee et les commandes principales sont executables de maniere deterministe.

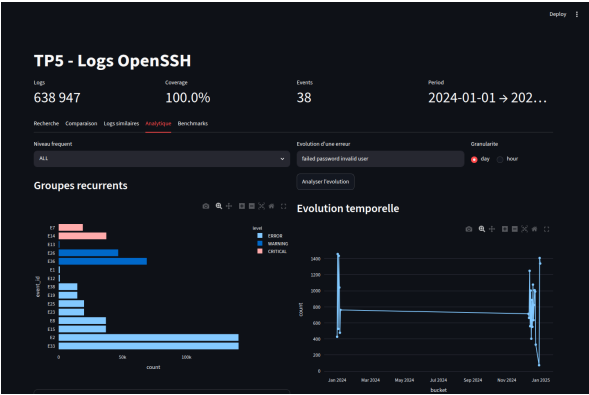
La qualite des resultats se verifie qualitativement. Pour une requete sur les echecs de mot de passe, les premiers resultats appartiennent aux templates `Failed password` ou



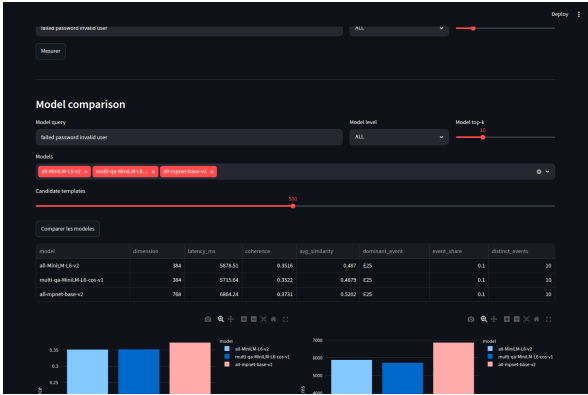
Recherche sémantique



Comparaison sémantique/mots-clés



Analytique des erreurs



Comparaison de modèles

Figure 1: Extraits de l'interface de visualisation.

authentication failure. Pour une requete sur les tentatives d'intrusion, les resultats font apparaitre les messages de type **POSSIBLE BREAK-IN ATTEMPT** ou utilisateurs invalides. Pour les analyses frequentes, les evenements les plus representes correspondent aux occurrences indiquees par LogHub.

La performance depend de la machine. Le pretraitement Spark du dataset complet reste raisonnable sur un poste local. La vectorisation par templates distincts reduit fortement le temps de calcul, car elle evite de traiter plusieurs centaines de milliers de messages quasi identiques. La recherche pgvector avec HNSW permet ensuite une latence compatible avec une interface interactive. Les mesures ajoutees permettent de documenter cette performance avec des chiffres reproductibles depuis l'API, la CLI ou l'interface web.

16 Tests et validation

Les tests unitaires valident le parsing OpenSSH, la normalisation des messages, la classification de niveau, la conversion des vecteurs au format pgvector et le parsing de la CLI. Les tests d'integration se font avec le pipeline limite, car ils impliquent Spark, PostgreSQL et le modele d'embedding.

Les points verifies sont :

- le pretraitement Spark produit les sorties CSV et Parquet ;
- PostgreSQL contient les logs et templates charges ;
- la table `message_embeddings` couvre les messages normalises distincts ;
- l'index HNSW existe sur la table `message_embeddings` ;
- une requete semantique retourne des messages OpenSSH coherents ;
- la comparaison mots-cles/semantique retourne deux listes distinctes ;
- l'onglet benchmarks affiche latence, tailles disque, coherence top-k et comparaison de modeles ;
- l'interface web communique avec l'API locale.

17 Limites et ameliorations

Le dataset OpenSSH ne contient pas l'annee dans le timestamp. L'annee configuree est donc une hypothese technique pour permettre les analyses temporelles. Une extension possible serait de croiser les logs avec une source externe contenant l'annee exacte.

La vectorisation par template est efficace et adaptee au dataset LogHub. Pour un environnement de production avec des messages plus varies, on pourrait combiner deux niveaux de vecteurs : un vecteur de template pour le regroupement et un vecteur de message brut pour la recherche fine.

Enfin, l'index HNSW privilegie la vitesse de recherche. Pour certaines evaluations, une recherche exacte sans index pourrait servir de reference afin de mesurer le compromis entre performance et rappel.

18 Conclusion

La plateforme constitue une chaine complete de recherche semantique et analytique sur logs massifs. Spark assure le traitement batch, PostgreSQL et pgvector fournissent la base vectorielle indexee, Sentence-Transformers apporte la representation semantique, FastAPI expose les fonctionnalites et Streamlit fournit l'interface de visualisation.

La solution couvre les cas d'utilisation principaux : retrouver les logs similaires a une erreur critique, identifier les groupes d'erreurs frequentes, analyser leur evolution temporelle et comparer la recherche semantique avec une recherche par mots-cles. Elle reste modulaire, reproductible et adaptable a d'autres jeux de logs.